

QA Automation Agent

Architecture & Design Choices

Prepared for: Tri-POD QA Assignment

Author: Jitender

Date: June 2026

*This document explains the rationale behind every major
technology and architecture decision in the QA Agent.*

Table of Contents

- 1. Executive Summary
- 2. Browser Agent: Browser Use vs Stagehand
- 3. Database: Supabase vs SQLite
- 4. Backend: Cloudflare Workers vs FastAPI
- 5. API Framework: Hono
- 6. Dashboard: Next.js on Cloudflare Pages
- 7. AI Model Strategy: Claude Sonnet + Haiku
- 8. UI/UX Design System Choices
- 9. Integration Architecture
- 10. Cost Analysis & Budget

1. Executive Summary

This QA Automation Agent is designed to autonomously pull tickets from Jira, Google Docs, and Google Sheets, generate or fetch test cases using AI, classify them as frontend or backend tests, execute them against a target web application, and report results with screenshots and logs.

The architecture prioritizes three key principles:

- Zero infrastructure cost - Using free tiers of Cloudflare Workers, Supabase, and Cloudflare Pages
- Autonomous intelligence - Browser Use agent with Claude AI for self-directed exploration
- Easy deployment - One-command deploys, no Docker/VPS required

Technology Stack Overview

Layer	Technology	Cost
Agent Runtime	Python 3.12+ (local)	Free
AI Model	Anthropic Claude Sonnet/Haiku	\$60 budget
Browser Automation	Browser Use + Playwright	Free
Backend API	Cloudflare Workers + Hono	Free (100K req/day)
Database	Supabase PostgreSQL	Free (500MB)
File Storage	Supabase Storage	Free (1GB)
Dashboard	Next.js + Cloudflare Pages	Free
Styling	Tailwind CSS + shadcn/ui	Free

2. Browser Agent: Browser Use vs Stagehand

The assignment specifically mentions two open-source browser AI agents: Stagehand (browserbase/stagehand) and Browser Use (browser-use/browser-use). Both are LLM-driven browser automation frameworks built on Playwright.

CHOSEN: Browser Use

Alternative: Stagehand

- Python-native: The entire agent core is Python. Browser Use integrates without any bridge layer, while Stagehand is TypeScript-first with a secondary Python wrapper.
- Autonomous reasoning loop: The assignment requires exploring an unfamiliar app "the way a QA hire would." Browser Use excels at autonomous exploration with its observe-plan-act loop, while Stagehand is better for known, deterministic workflows.
- Massive community: 97K+ GitHub stars vs ~3.4K for Stagehand. More examples, faster bug fixes, better documentation, and battle-tested in production.
- Built-in cost tracking: `calculate_cost=True` tracks token spend per run. Critical when operating under a \$60 budget cap. Stagehand requires manual cost tracking.
- DOM-first approach: Browser Use reads DOM structure by default, saving 80-90% tokens compared to vision-only approaches. Only falls back to screenshots when needed.
- Zero cloud cost: Runs 100% locally with Playwright. Stagehand recommends Browserbase cloud (\$20+/mo) for production use.
- Built-in retry handling: `max_failures` parameter provides step-level retry with automatic backoff, perfect for flaky UI tests.

Feature	Browser Use	Stagehand
Language	Python (native)	TypeScript (Python wrapper)
GitHub Stars	97,400+	~3,400
Autonomy	Full agent loop	Hybrid code+NL
Cost Tracking	Built-in	Manual
Retry Handling	<code>max_failures</code> param	Self-healing cache
Cloud Required?	No	Recommended
Claude Support	Native	Native
Token Efficiency	DOM-first (80-90% savings)	CDP + caching

3. Database: Supabase vs SQLite

The agent needs to persist tickets, test cases, test results, and screenshots. Supabase provides a free PostgreSQL database with real-time subscriptions and file storage.

CHOSEN: Supabase (PostgreSQL)	Alternative: SQLite
-------------------------------	---------------------

- Cloud-accessible: The dashboard (deployed on Cloudflare Pages) can query Supabase directly via its JavaScript client. SQLite is a local file that cannot be accessed from a web app without a server.
- Real-time subscriptions: Supabase Realtime enables live updates in the dashboard as tests execute. No need to build WebSocket infrastructure.
- File storage: Screenshots can be uploaded to Supabase Storage (1GB free) and served via CDN URLs. SQLite cannot store files.
- Shareable with evaluators: Just add their email to get access to the database. With SQLite, you would need to share the file manually.
- Free tier is generous: 500MB database, 1GB file storage, 50K monthly active users, unlimited API requests.
- Auto-generated REST API: Supabase creates REST endpoints for all tables automatically, reducing backend code.

Feature	Supabase	SQLite
Dashboard access	Direct via JS client	Needs server proxy
Real-time updates	Built-in	Requires polling
File storage	1GB free	Local disk only
Shareable	Add email	Share file
Free tier	500MB DB	Unlimited (local)
Setup complexity	Create project (2 min)	Zero config

4. Backend: Cloudflare Workers vs FastAPI

The API layer serves as the bridge between the Python agent (running locally) and the Next.js dashboard (deployed globally). Cloudflare Workers provides a serverless edge runtime with zero cold starts.

CHOSEN: Cloudflare Workers

Alternative: FastAPI (Python)

- Zero deployment cost: Free tier includes 100K requests/day, which is far more than this project needs. FastAPI would require a VPS (\$5+/mo) or Docker container.
- One-command deploy: "wrangler deploy" pushes the API to 300+ edge locations globally. FastAPI needs Docker, a VPS, SSH, nginx, SSL certificates.
- Zero cold starts: Workers run on Cloudflare edge network with instant startup. FastAPI on a serverless platform would have 500ms-2s cold starts.
- Automatic HTTPS: SSL certificates are provisioned automatically. No manual cert management.
- Global CDN: API responses are served from the nearest edge location. FastAPI on a single VPS means latency from one region.
- The Python agent runs locally anyway: Since Browser Use requires a local browser, the agent must run on the developer machine. The API layer is thin (just CRUD on Supabase), so Workers is perfect.

Feature	Cloudflare Workers	FastAPI
Cost	Free (100K req/day)	\$5+/mo VPS
Deploy	1 command	Docker + VPS
Cold starts	0ms	500ms-2s
HTTPS	Automatic	Manual certs
Global CDN	Yes (300+ locations)	Single region
Scaling	Automatic	Manual

5. API Framework: Hono

Hono is a lightweight (14KB), ultrafast web framework designed specifically for Cloudflare Workers and edge runtimes.

Why Hono?

- Cloudflare-native: Built for Workers from the ground up. First-class support for env bindings, KV, D1, etc.
- Express-like API: Familiar middleware pattern (app.get, app.post, app.use) minimizes learning curve.
- 14KB bundle: Tiny footprint keeps Workers within the free tier CPU limits (10ms).
- Built-in middleware: CORS, auth, rate limiting, validation all included. No external deps needed.
- TypeScript-first: Full type safety with excellent inference.

6. Dashboard: Next.js on Cloudflare Pages

The dashboard provides real-time visibility into test execution, results, and agent activity.

- Next.js 15: Latest version with React 19, server components, and app router.
- Cloudflare Pages: Free hosting with automatic builds from Git. Global CDN, preview deployments.
- Supabase Realtime: Dashboard subscribes to database changes for live updates. No WebSocket server needed.
- Tailwind CSS + shadcn/ui: Pre-built, accessible components with dark mode support.

7. AI Model Strategy

With a \$60 Anthropic API budget, strategic model selection is critical. The agent uses different Claude models for different tasks based on complexity vs cost.

Task	Model	Est. Cost/Call	Why
Browser Agent	Claude Sonnet	~\$0.15	Complex DOM reasoning
Test Generation	Claude Sonnet	~\$0.16	Structured output needed
Classification	Claude Haiku	~\$0.02	Simple categorization
Report Generation	Claude Haiku	~\$0.05	Template-based
App Discovery	Claude Sonnet	~\$0.17	Understanding UI flows

Budget Breakdown

Use Case	Est. Calls	Est. Cost
Browser agent (frontend tests)	~200	\$30
Test case generation	~50	\$8
App discovery/exploration	~30	\$5
Test classification	~100	\$2
Report generation	~20	\$1
Development/debugging	~50	\$8
Buffer	-	\$6
TOTAL		\$54 (of \$60)

8. UI/UX Design System Choices

The design system was generated using the ui-ux-pro-max skill, which analyzed 161 color palettes, 86 UI styles, 57 font pairings, and 1,776 design specifications to recommend the optimal visual system for a QA testing dashboard.

Product Type Match: Developer Tool / IDE

- Primary style: Dark Mode (OLED) + Minimalism - Industry standard for developer tools
- Dashboard pattern: Real-Time Monitor + Terminal aesthetic
- Color theme: "Code dark + run green" - Deep slate backgrounds with green success indicators
- Typography: Inter (UI) + JetBrains Mono (code/data) - The standard dev-tool pairing

Color Palette

Token	Value	Purpose
Background	#0F172A	Deep slate - reduces eye strain
Card Surface	#1B2336	Elevated elements
Pass/Success	#22C55E	Green - universal "go/pass"
Fail/Error	#EF4444	Red - universal "stop/fail"
Warning	#F59E0B	Amber - attention needed
In Progress	#3B82F6	Blue - active/running
Text Primary	#F8F8F8	High contrast on dark
Text Secondary	#9A9A9A	Muted for labels

Key UX Rules Applied

- WCAG AA contrast: 4.5:1 minimum (7:1+ for dark mode text)
- No emoji as structural icons - using Lucide SVG icons
- 44x44px minimum touch/click targets
- 150-300ms animation timing with prefers-reduced-motion support
- 4pt/8dp spacing system throughout
- Virtualized lists for 50+ test cases
- Never pure #000000 backgrounds (OLED smear prevention)

9. Integration Architecture

The agent integrates with four external services. Here is why each integration was designed the way it is.

Jira REST API v3

- Authentication: Basic Auth with API token (simplest, sufficient for a demo)
- JQL queries for flexible ticket filtering by project, status, label
- Results posted as comments on tickets for traceability

Google Sheets (gsread library)

- Primary shareable output: Evaluators can directly open and inspect results
- Color-coded pass/fail cells for visual scanning
- Service Account auth (no OAuth popup needed)

Google Docs

- Detailed narrative reports with embedded screenshots
- Used for ticket descriptions and rich test documentation

Supabase Realtime

- Dashboard subscribes to INSERT events on test_results table
- Live KPI updates as tests complete - no polling or WebSocket server needed
- Screenshot URLs served from Supabase Storage CDN

10. Cost Analysis

Total Infrastructure Cost: \$0/month

Service	Free Tier Limit	Our Usage (Est.)
Cloudflare Workers	100K req/day	~500 req/day
Cloudflare Pages	Unlimited sites	1 site
Supabase DB	500MB	~50MB
Supabase Storage	1GB	~200MB screenshots
Supabase Auth	50K MAU	Not used (API key)
Google Sheets API	300 req/min	~50 req/run
Google Docs API	300 req/min	~10 req/run

The only cost is the \$60 Anthropic API credits provided for the assignment. All infrastructure services operate within their generous free tiers.